

Statistical and Machine Learning

Evaluation of model performance

Exercises and Model Answers

Exercise 1 – Confusion Matrix and Basic Metrics

A binary classifier is evaluated on 200 patients (cancer vs healthy). The results are: TP = 40 (true positives, i.e., a person having cancer was predicted as such), FP = 10 (false positives), FN = 20 (false negatives), TN = 130 (true negatives).

1. Fill in the confusion matrix for this classifier.

Model answer:

	Predicted Positive	Predicted Negative
Actually Positive	TP = 40	FN = 20
Actually Negative	FP = 10	TN = 130

Total = 40 + 10 + 20 + 130 = 200.

2. Compute the **accuracy**, **sensitivity**, **specificity**, and **precision**.

Model answer:

$$\text{Accuracy} = \frac{TP + TN}{200} = \frac{40 + 130}{200} = \frac{170}{200} = 0.85$$

$$\text{Sensitivity} = \frac{TP}{TP + FN} = \frac{40}{40 + 20} = \frac{40}{60} \approx 0.667$$

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{130}{130 + 10} = \frac{130}{140} \approx 0.929$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{40}{40 + 10} = \frac{40}{50} = 0.80$$

3. Suppose this classifier is used to screen for cancer. Which metric is most important — sensitivity or specificity — and why? In other words, which metric should we maximise?

Model answer: Sensitivity (TPR) is more important in cancer screening. A false negative (predicting a cancer patient as healthy) means a patient goes untreated and may suffer fatal consequences. Missing a true case is far costlier than a false alarm, so we prioritise minimising FN, i.e., maximising sensitivity. A false positive means an unnecessary follow-up test — inconvenient and costly, but not life-threatening.

4. A second classifier has $TP = 55$, $FP = 30$, $FN = 5$, $TN = 110$. Compute its accuracy. Explain why accuracy alone is insufficient to compare the two classifiers in this context.

Model answer: Both classifiers achieve almost the same accuracy ($165/200 = 82.5\%$), but they behave very differently. Classifier 2 has sensitivity $= 55/60 \approx 0.917$ (much higher) but specificity $= 110/140 \approx 0.786$ (lower). For cancer screening, Classifier 2 is clearly preferable because it misses far fewer cancer patients ($FN = 5$ vs $FN = 20$), at the cost of more false alarms. Accuracy weights all errors equally and ignores class imbalance and the asymmetric costs of different error types.

Exercise 2 – F1 Score and the Precision–Recall Trade-off

1. Using the counts from Exercise 1 ($TP = 40$, $FP = 10$, $FN = 20$, $TN = 130$), compute the **F1 score**.

Model answer:

```
TP <- 40; FP <- 10; FN <- 20; TN <- 130

precision <- TP / (TP + FP)
sensitivity <- TP / (TP + FN)
F1 <- 2 * precision * sensitivity / (precision + sensitivity)

cat(sprintf("Precision    = %.3f\n", precision))

## Precision    = 0.800

cat(sprintf("Sensitivity  = %.3f\n", sensitivity))

## Sensitivity  = 0.667

cat(sprintf("F1 score     = %.3f\n", F1))

## F1 score     = 0.727
```

From the counts: Precision $P = 0.80$, Sensitivity $S = 0.667$.

$$F1 = \frac{2 \times P \times S}{P + S} = \frac{2 \times 0.80 \times 0.667}{0.80 + 0.667} = \frac{1.067}{1.467} \approx 0.727$$

Intuition behind the F1 score:

Precision and sensitivity each tell only half the story. A model that flags *every* patient as septic achieves perfect sensitivity (it never misses a case) but near-zero precision (almost every flag is a false alarm). A model that only flags patients it is almost certain about achieves high precision but misses most true cases. Neither extreme is useful.

F1 combines the two into a single number by taking their **harmonic mean**. The harmonic mean is always pulled toward the *lower* of the two values, which is exactly what we want: a model cannot hide a poor sensitivity behind a great precision, or vice versa. Concretely, if either metric collapses to zero, F1 collapses to zero too — the arithmetic mean would not behave this way.

To see why this matters here: our model has precision 0.80 and sensitivity 0.667. The arithmetic mean would give 0.733, which sounds reasonable. The harmonic mean (F1) gives 0.727 — slightly lower, because it penalises the weaker of the two metrics (sensitivity). The gap is small in this case, but becomes stark when the two metrics are very far apart.

A useful way to remember it: **F1 rewards balance**. A model with $P = 0.90$, $S = 0.50$ scores $F1 = 0.643$, while a model with $P = 0.70$, $S = 0.70$ scores $F1 = 0.700$ — the balanced model wins, even though its individual metrics look less impressive.

2. In general, lowering the classification threshold increases recall but decreases precision. Explain intuitively why this is the case.

Model answer: Lowering the threshold means the classifier labels more observations as positive (a lower score is sufficient to trigger a positive prediction). This has two effects:

- More true positives are captured (fewer missed cases), so sensitivity = $TP/(TP + FN)$ **increases**.
- But more false positives are also introduced (healthy patients incorrectly flagged), so precision = $TP/(TP + FP)$ **decreases**.

At the extreme of threshold = 0, everything is labelled positive: sensitivity = 1 but precision equals the base rate.

The code below illustrates this trade-off by simulating predictions at varying thresholds:

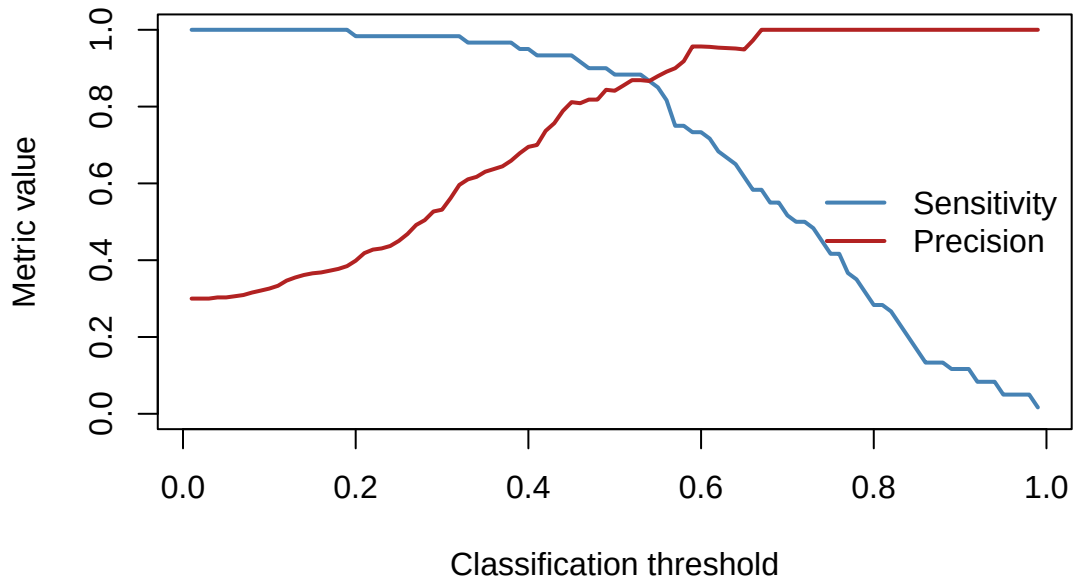
```
set.seed(42)
n      <- 200
labels <- c(rep(1, 60), rep(0, 140))      # 60 positives, 140 negatives
scores <- ifelse(labels == 1,
                  rbeta(60, 5, 2),        # positives score higher
                  rbeta(140, 2, 5))

thresholds <- seq(0.01, 0.99, by = 0.01)
precision_v <- numeric(length(thresholds))
sensitivity_v <- numeric(length(thresholds))

for (i in seq_along(thresholds)) {
  pred <- as.integer(scores >= thresholds[i])
  tp  <- sum(pred == 1 & labels == 1)
  fp  <- sum(pred == 1 & labels == 0)
  fn  <- sum(pred == 0 & labels == 1)
  precision_v[i] <- ifelse(tp + fp == 0, NA, tp / (tp + fp))
  sensitivity_v[i] <- ifelse(tp + fn == 0, NA, tp / (tp + fn))
}

plot(thresholds, sensitivity_v, type = "l", col = "steelblue", lwd = 2,
     ylim = c(0, 1), xlab = "Classification threshold",
     ylab = "Metric value", main = "Precision vs Sensitivity across thresholds")
lines(thresholds, precision_v, col = "firebrick", lwd = 2)
legend("right", legend = c("Sensitivity", "Precision"),
     col = c("steelblue", "firebrick"), lwd = 2, bty = "n")
```

Precision vs Sensitivity across thresholds



As the threshold rises, precision climbs while sensitivity falls — the classic trade-off.

3. Give one real-world scenario where you would prefer **high recall** over high precision, and one where you would prefer **high precision** over high recall.

Model answer:

- **High recall preferred:** Cancer screening, fraud detection in banking, or any setting where missing a true positive has severe consequences. Accepting more false alarms is tolerable.
- **High precision preferred:** Email spam filtering (we do not want to incorrectly bin legitimate emails), or a news recommendation system (we want to show only genuinely relevant articles, not a flood of irrelevant ones).

Exercise 3 – ROC Curve and AUC

1. Explain what the ROC curve plots on each axis, and what the role of the threshold t is.

Model answer: The ROC curve plots the **False Positive Rate** ($FPR = FP/(FP + TN)$) on the x -axis against the **True Positive Rate** ($TPR = TP/(TP + FN)$) on the y -axis. A classifier outputs a continuous score $\hat{y}_i \in [0, 1]$. For each threshold $t \in [0, 1]$, observations with $\hat{y}_i \geq t$ are labelled positive. As t varies from 1 to 0, both TPR and FPR change, tracing out the ROC curve. The curve therefore visualises classifier performance across all possible thresholds simultaneously.

2. What does an AUC of 0.5 correspond to, and what does an AUC of 1.0 correspond to? What is the AUC of a classifier that always predicts the majority class?

Model answer:

- **AUC = 0.5:** The classifier performs no better than random guessing — its ROC curve follows the diagonal. It cannot distinguish positives from negatives.
- **AUC = 1.0:** A perfect classifier — it achieves $TPR = 1$ and $FPR = 0$ simultaneously; every positive is ranked above every negative.

- **Always predicting the majority class:** Such a classifier looks at the training data, notices which class is more frequent, and then predicts that class for every observation regardless of individual features. Because every observation receives the same score, varying the threshold produces only two points on the ROC curve: (0,0) when the threshold is so high nothing is flagged, and (1,1) when the threshold is so low everything is flagged. The ROC curve collapses to the diagonal, giving **AUC = 0.5** — the same as random guessing.

Note: AUC = 0 would require a perversely *perfect* classifier in the wrong direction — one that ranks every negative above every positive. That takes skill, just entirely misdirected.

Concrete example:

Imagine 5 patients: 2 with sepsis (positive) and 3 without (negative). A doctor (our “classifier”) inspects the training data, finds that “no sepsis” is the majority class, and assigns score 0 to every patient.

```
patients <- data.frame(
  Patient      = c("A", "B", "C", "D", "E"),
  `True label` = c("Positive", "Positive", "Negative", "Negative", "Negative"),
  `Predicted score` = c(0, 0, 0, 0, 0),
  check.names  = FALSE
)
knitr::kable(patients, align = "ccc")
```

Patient	True label	Predicted score
A	Positive	0
B	Positive	0
C	Negative	0
D	Negative	0
E	Negative	0

Calculations of the ROC curve:

```
thresh_df <- data.frame(
  `Predicted positive` = c("Nobody", "Everyone"),
  TP = c(0, 2), FP = c(0, 3),
  TPR = c("0/2 = 0.00", "2/2 = 1.00"),
  FPR = c("0/3 = 0.00", "3/3 = 1.00"),
  check.names = FALSE
)
knitr::kable(thresh_df, align = "cccccc")
```

Predicted positive	TP	FP	TPR	FPR
Nobody	0	0	0/2 = 0.00	0/3 = 0.00
Everyone	2	3	2/2 = 1.00	3/3 = 1.00

The ROC curve is just the two points (0,0) and (1,1) joined by a straight line — the diagonal. The area under this line is a right triangle with base 1 and height 1:

$$\text{AUC} = \frac{1}{2} \times 1 \times 1 = 0.5$$

The classifier used the data to pick a strategy, but that strategy discards all individual patient information. It cannot rank a positive above a negative at any threshold, so it is indistinguishable from a random coin flip.

```

# ROC curve for the majority-class classifier
fpr_maj <- c(0, 1)
tpr_maj <- c(0, 1)

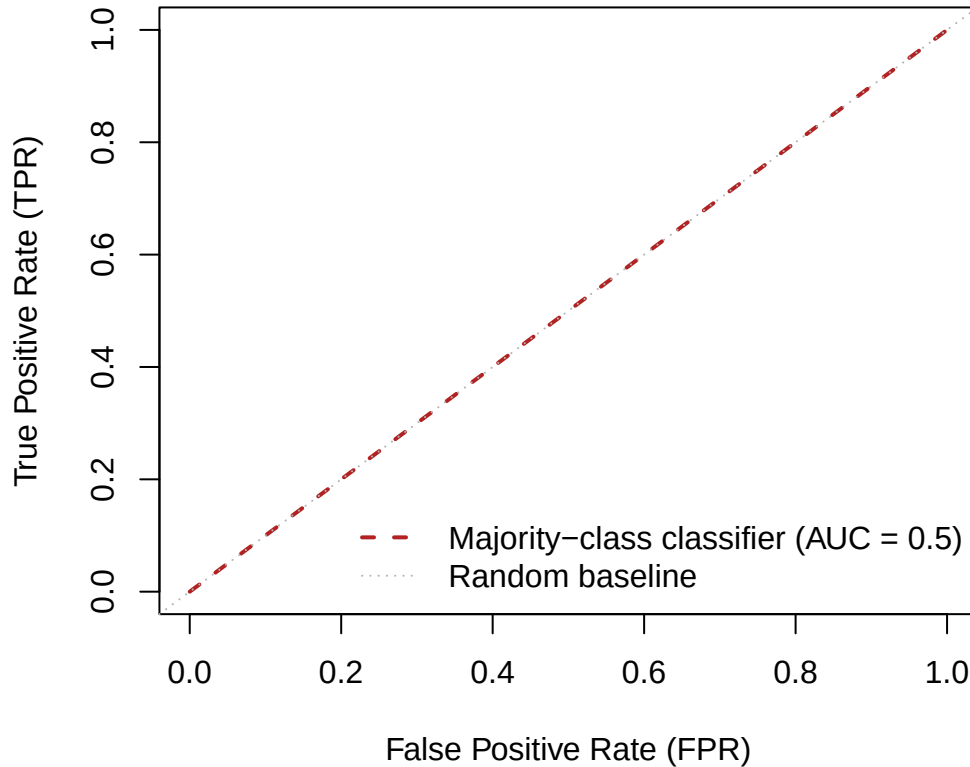
# For contrast: a decent classifier on the same 5-patient toy example
# Scores: positives get 0.8 and 0.7; negatives get 0.6, 0.4, 0.2
labels_toy <- c(1, 1, 0, 0, 0)
scores_toy <- c(0.8, 0.7, 0.6, 0.4, 0.2)

thresholds <- sort(unique(scores_toy), decreasing = TRUE)
roc_pts <- data.frame(fpr = 0, tpr = 0)
for (t in thresholds) {
  pred <- as.integer(scores_toy >= t)
  tp <- sum(pred == 1 & labels_toy == 1)
  fp <- sum(pred == 1 & labels_toy == 0)
  roc_pts <- rbind(roc_pts,
                  data.frame(fpr = fp / sum(labels_toy == 0),
                             tpr = tp / sum(labels_toy == 1)))
}
roc_pts <- rbind(roc_pts, data.frame(fpr = 1, tpr = 1))

plot(fpr_maj, tpr_maj, type = "l", col = "firebrick", lwd = 2, lty = 2,
     xlim = c(0, 1), ylim = c(0, 1),
     xlab = "False Positive Rate (FPR)",
     ylab = "True Positive Rate (TPR)",
     main = "ROC curves: majority-class vs. a better classifier")
abline(0, 1, col = "grey70", lty = 3)
legend("bottomright",
      legend = c("Majority-class classifier (AUC = 0.5)",
                  "Random baseline"),
      col = c("firebrick", "grey70"),
      lwd = c(2, 1),
      lty = c(2, 3),
      bty = "n")

```

ROC curves: majority-class vs. a better classifier



The dashed red line is the majority-class classifier — it hugs the diagonal and contributes no useful discrimination. The solid blue line shows a classifier that actually separates positives from negatives, bulging toward the top-left corner and achieving $\text{AUC} > 0.5$.

3. Two models A and B have $\text{AUC} = 0.85$ and $\text{AUC} = 0.78$ respectively. Model A is preferred overall, but a doctor requires that the false positive rate never exceed 5%. Can AUC alone determine which model to use? Explain.

Model answer: No. AUC is a global summary of the ROC curve across all thresholds. It is entirely possible that model B has a higher TPR than model A in the low-FPR region ($\text{FPR} \leq 0.05$), even though model A dominates on average. To answer the doctor's question one must inspect the ROC curves directly at $\text{FPR} = 0.05$, or use the partial AUC (pAUC) restricted to that region. A higher overall AUC does not guarantee better performance under specific operating constraints.

Exercise 4 – Regression Loss Functions

1. Write down the L_1 , L_2 , and Huber loss functions.

Model answer:

$$L_1 = \sum_{i=1}^n |y_i - \hat{y}_i|$$
$$L_2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$L_\delta(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{if } |y - f(x)| \leq \delta \\ \delta(|y - f(x)| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

2. A dataset contains 100 observations with small residuals, plus 5 highly influential observations. Sketch (or describe) what happens to the fitted regression line when you use L_2 vs L_1 loss, and explain the difference.

Model answer: Under L_2 **loss**, the 5 highly influential observations are penalised quadratically – their contribution to the objective grows as the square of the residual. The fitted line is therefore pulled strongly toward the influential observations, producing a slope and intercept that differ substantially from the underlying pattern in the 100 clean points.

Under L_1 **loss**, each residual contributes linearly regardless of its size. The influential observations are down-weighted relative to L_2 , so the fitted line is much closer to the line that describes the 100 clean points. This robustness is the main practical advantage of L_1 in the presence of contaminated data (also illustrated in the R simulation in the lecture slides).

3. The Huber loss has a tuning parameter δ . What happens to the Huber loss as $\delta \rightarrow \infty$? What happens as $\delta \rightarrow 0$?

Model answer:

- As $\delta \rightarrow \infty$: all residuals satisfy $|r| \leq \delta$, so the Huber loss reduces to $\frac{1}{2}r^2$ everywhere, i.e., it converges to the L_2 **loss**.
 - As $\delta \rightarrow 0$: the quadratic region shrinks to zero, so the Huber loss converges to $\delta|r|$, i.e., (up to a factor δ) it converges to the L_1 **loss**.
4. Give one practical reason why L_2 is more commonly used than L_1 despite L_1 being more robust to influential observations.

Model answer: L_2 is **differentiable everywhere**, so standard gradient-based optimisation (e.g., normal equations in OLS, gradient descent in ML) can be applied directly. L_1 has a non-differentiable kink at 0, requiring subgradient methods or specialised solvers. L_2 also has a closed-form solution in linear regression, making it computationally and analytically more convenient.

Exercise 5 – Cross-Validation

1. Describe the K -fold cross-validation procedure in your own words. What is the purpose of holding out one fold at a time?

Model answer: The data are randomly divided into K equal parts (folds). In each of K iterations, one fold is held out as the validation set and the model is trained on the remaining $K - 1$ folds. The trained model then produces predictions on the held-out fold, and a prediction error is computed. After all K iterations, the errors are averaged to produce the CV estimate.

The purpose of holding out data is to simulate how the model will perform on unseen observations. Because the held-out fold was not used for training, the prediction error it yields is an approximately unbiased estimate of the true generalisation error — unlike the training error, which systematically underestimates it due to overfitting.

2. Write down the cross-validation estimate of prediction error $CV_{(K)}$.

Model answer:

$$CV_{(K)} = \frac{1}{K} \sum_{k=1}^K CV_k, \quad \text{where} \quad CV_k = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \left(y_i - \hat{y}_i^{(-k)} \right)^2$$

and $\hat{y}_i^{(-k)}$ is the prediction for observation i from the model trained without fold k .

3. Consider $K = n$ (leave-one-out CV) and $K = 2$. Describe one advantage and one disadvantage of each.

Model answer:

Leave-one-out CV ($K = n$):

- Advantage: Each training set uses $n - 1$ observations, so the model is trained on almost all the data — low bias in the CV estimate.
- Disadvantage: Requires fitting the model n times, which is computationally expensive for large n or slow-to-fit models.

2-fold CV ($K = 2$):

- Advantage: Only two model fits required — very fast.
- Disadvantage: Each model is trained on only half the data, so the training sets are much smaller than the full dataset, leading to higher bias (the CV error overestimates the true test error). High variance across splits is also common.

Follow-up 1: Why might larger K be better?

The main reason is that larger K means each fitted model is trained on a larger share of the data.

In K -fold cross-validation, each training set has approximately $n \left(1 - \frac{1}{K}\right)$ observations. As K increases, $\frac{1}{K}$ decreases, so $1 - \frac{1}{K}$ gets closer to 1. Hence the training-set size gets closer to n .

This matters because the final model is usually trained on the full dataset. Therefore, the CV estimate is most directly relevant when the models fitted inside CV resemble the final fitted model. With larger K , the models used inside CV are trained on almost the same amount of data as the final model, so their expected test error should be closer to the final model's expected test error.

For example, when $K = 2$, each model is trained on only $\frac{n}{2}$ observations. This can make the CV estimate pessimistic, because models trained on half the data may perform worse than the final model trained on all n observations.

By contrast, when $K = n$, each model is trained on $n - 1$ observations, which is nearly the full dataset. This is why leave-one-out CV typically has low bias as an estimate of the final model's test error.

Thus, in the bias sense, larger K , leads to larger training sets which results in less pessimistic CV error.

The caveat is that this does not automatically imply lower variance or lower mean squared error. Larger K reduces the training-size bias, but the variance of the CV estimate can behave more subtly because the fitted models across folds are dependent.

Follow-up 2: What if K grows with n ?

Now let the number of folds depend on the sample size, so write $K = K_n$. Suppose $K_n \rightarrow \infty$ and $K_n = o(n)$. This regime is attractive because it gives two useful properties at the same time.

First, the training-set size is approximately $n \left(1 - \frac{1}{K_n}\right)$. Since $K_n \rightarrow \infty$, we have $\frac{1}{K_n} \rightarrow 0$, and therefore $n \left(1 - \frac{1}{K_n}\right) \sim n$. So the models fitted inside cross-validation are trained on an asymptotically full fraction of the data. This reduces the pessimistic bias discussed above.

Second, the validation-fold size is approximately $\frac{n}{K_n}$. Because $K_n = o(n)$, we have $\frac{K_n}{n} \rightarrow 0$, which implies $\frac{n}{K_n} \rightarrow \infty$. So each validation fold also grows in size. This avoids the extreme leave-one-out situation where each validation set contains only one observation.

Hence this asymptotic setting balances the two goals: $K_n \rightarrow \infty \Rightarrow$ training sets become nearly full-size while $K_n = o(n) \Rightarrow$ validation folds still grow.

Therefore, in this setting, “more folds are better” in the sense that increasing K_n reduces the bias caused by training on less than the full sample, while still keeping validation folds large enough to provide stable error estimates.

The caveat is the same as before: this is mainly a bias argument. It does not prove that the largest possible K_n gives the best finite-sample mean squared error, because variance and dependence across folds also matter.

Follow-up 3: Selecting K in K -fold cross-validation

In practice, K is chosen to balance bias, variance, and computational cost. Larger K means that each model is trained on a larger fraction of the data, $n \left(1 - \frac{1}{K}\right)$, so the CV estimate is usually less pessimistically biased. However, larger K also requires fitting more models, making the procedure more expensive.

Common default choices are $K = 5$ or $K = 10$. These values are popular because they give a good compromise: the training sets are large enough to resemble the full dataset, but the number of model fits is still manageable. For example, 5-fold CV trains on $0.8n$ observations, while 10-fold CV trains on $0.9n$.

Leave-one-out CV, where $K = n$, has low bias because each model is trained on $n - 1$ observations, but it can be computationally expensive and does not necessarily have the lowest variance or mean squared error. A practical rule is to use $K = 5$ or $K = 10$ in most settings, use larger K when n is small and models are cheap to fit, and use smaller K when n is large or model fitting is expensive.

Exercise 6 – Bias–Variance Trade-off and Double Descent

1. State the bias-variance decomposition of the MSE. Define each term.

Model answer:

$$\text{MSE}(\hat{\theta}) = \text{Bias}^2(\hat{\theta}) + \text{Var}(\hat{\theta})$$

- $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta_0$: systematic error — how far on average the estimator is from the truth.
 - $\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$: variability of the estimator across different training sets — how sensitive it is to the particular sample.
2. A polynomial regression model is fitted with degrees $d = 1, 2, \dots, 15$. Sketch (or describe) the expected shape of the training error and test error curves as d increases. What is the name of the phenomenon at high d ?

Model answer: As d increases:

- **Training error** decreases monotonically — higher-degree polynomials can fit the training data ever more closely.
- **Test error** follows a U-shape: it decreases initially (the model learns the true signal), reaches a minimum at some intermediate complexity, and then increases again (the model begins fitting noise).

The increase in test error at high complexity is called **overfitting**. The model has low bias but high variance — it is too sensitive to the specific training sample and fails to generalise.

3. Describe the **double descent** phenomenon. At what point does the second descent begin, and what drives it?

Model answer: Double descent extends the classical U-shaped test error curve into a three-stage pattern as model complexity (e.g., number of parameters p) increases:

1. Underparameterised regime ($p < n$): Test error decreases as the model gains capacity (classical bias-variance trade-off).
2. Interpolation threshold ($p \approx n$): Test error spikes — the model just becomes flexible enough to perfectly fit the training data, but the fit is unstable and highly variable.
3. Overparameterised regime ($p \gg n$): Test error decreases again. Among all models that interpolate the training data, the optimiser (e.g., gradient descent) tends to select **minimum-norm** or **smooth** solutions via implicit regularisation. These solutions have lower effective variance and generalise better.

The key insight is that model size and effective complexity are not the same: very large models may have less effective complexity than models near the interpolation threshold.

Exercise 7 – Bootstrap

1. Describe the bootstrap procedure for estimating the standard error of an estimator $\hat{\theta}$. What is the key difference from classic repeated sampling?

Model answer: Given a dataset of n observations:

1. Draw a bootstrap sample $\mathcal{D}_b^*$ of size n **with replacement** from the original data.
2. Compute the estimator on this sample: $\hat{\theta}_b^*$.
3. Repeat steps 1–2 for $b = 1, \dots, B$ (e.g., $B = 1000$).
4. Estimate the standard error as:

$$\widehat{SE}(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}_b^* - \bar{\hat{\theta}})^2}$$

Key difference from classic repeated sampling: In classic repeated sampling one draws fresh independent samples from the true (unknown) population. In the bootstrap, the original data itself serves as a surrogate for the population, and resampling with replacement mimics drawing new samples. This is feasible in practice precisely because the true population is unknown.

2. In the portfolio example, $\bar{\alpha} = 0.666$ and $\widehat{SD}(\hat{\alpha}) = 0.076$. Interpret these two numbers.

Model answer: $\bar{\alpha} = 0.666 \approx \frac{2}{3}$ is the bootstrap estimate of the expected value of $\hat{\alpha}$ — it is close to the true value of 0.667, indicating that the estimator is approximately unbiased. $\widehat{SD}(\hat{\alpha}) = 0.076$ is the bootstrap estimate of the standard error of $\hat{\alpha}$: across repeated samples of the same size, the estimate of the optimal portfolio weight α would typically vary from its mean by about ± 0.076 . This quantifies the sampling uncertainty (reliability) of the estimator.

3. Name one assumption under which bootstrap resampling faithfully mimics population sampling, and one situation where it might fail.

Model answer:

- **Assumption:** The observations are **i.i.d.** (independent and identically distributed). Under i.i.d., the empirical distribution of the data converges to the true population distribution, so resampling from the data approximates sampling from the population.
- **Failure case:** The bootstrap can fail when observations are **dependent** (e.g., time series data) – standard resampling destroys the temporal structure. It can also fail for estimators at the boundary of the parameter space, or for very heavy-tailed distributions where the empirical distribution is a poor approximation to the population.